CYBERNAUT · ML-07 FINAL REPORT DRAFT

PhishShift Message Risk Scanner

Student: _____ Team/Student ID: _____
Mentor: _____ Date: _____

Abstract

PhishShift is an educational message-risk scanner designed to compare a rule-based baseline with machine-learning approaches for controlled email/SMS training messages. The prototype removes normalized duplicate templates before splitting, keeps campaign groups separate between training and test data, and compares rules, TF-IDF with Logistic Regression, and TF-IDF with Random Forest. Its local Flask interface displays a model-estimated phishing probability, a manual-review abstain band, and simple text-based risk cues. It does not open links, fetch remote content, execute attachments, or collect credentials. The included dataset consists of locally authored synthetic examples, so it is suitable for repeatable prototype demonstrations but not a substitute for a licensed real-world corpus. All metrics must be interpreted in light of this limitation and should be updated after the student reruns the project and evaluates the mentor-supplied unseen campaign.

1. Problem and objectives

The project addresses the risk that keyword-heavy training can fail on unfamiliar wording and can falsely flag legitimate urgent messages. Objectives are to remove duplicate templates, compare three approaches, test on unseen campaigns, include abstention and explanations, and report false positives and robustness drift.

2. Literature and sources

The package's current corpus is synthetic and locally authored; it does not copy a public corpus. Before final submission, add at least three sources reviewed by the student and verify licensing/permissions if a public dataset is added. Suggested starting points: (1) scikit-learn documentation for text feature extraction and model evaluation; (2) NIST guidance on phishing and security awareness; (3) a clearly licensed phishing email dataset or security-awareness corpus approved by the mentor. Record exact titles, URLs, access dates, and licenses in the final version.

3. Data and EDA

The CSV contains 720 generated examples before deduplication and includes message ID, channel, subject/title, body, inert URL-feature description, sender type, label, campaign group, collection date, topic, and provenance. No real personal information or active URLs are intentionally included. The training script writes a duplicate log, split files, split summary, model-comparison metrics, held-out predictions, and robustness test results under outputs/.

4. Design and methods

Workflow: synthetic corpus → schema loading → normalization and duplicate removal → campaign-group holdout → rule baseline and two ML models → metric comparison and error review → local Flask UI. The main text classifier uses TF-IDF word uni/bi-grams and Logistic Regression. The third model is a TF-IDF + Random Forest pipeline. A campaign-disjoint assertion stops training if any campaign group overlaps.

5. Metrics and results

Run `python train_model.py` and use outputs/model_comparison.csv for actual values. Report accuracy alongside phishing precision, recall, F1 and false-positive rate (FP / (FP + TN)); accuracy alone is insufficient. Review outputs/heldout_predictions.csv and outputs/robustness_tests.csv. Do not copy generic values into this report: record your own run, identify the selected model, and discuss at least three errors.

6. Safety, limitations and ethics

The application processes pasted text locally and does not store scans or contact remote URLs. It warns users not to enter real secrets. The probability is model-estimated and not a calibrated guarantee. Phrase matching in explanations is a simple aid, not a causal explanation. Synthetic examples can create optimistic results; unseen mentor data is required for a stronger generalization claim.

7. Conclusion and future improvements

This prototype provides a repeatable baseline for message triage, group-based evaluation, model comparison, abstention, and controlled robustness testing. Future work can add an approved licensed corpus, richer sender/header features, calibration, subgroup error analysis, and a documented human-review workflow without fetching message links.

Appendix: reproducibility checklist

- Install dependencies from requirements.txt
- Run train_model.py and save console output
- Confirm campaign_overlap is empty
- Inspect duplicate log and class distribution
- Review false positives and false negatives
- Run the web UI and capture real screenshots
- Test AT-01, AT-02, AT-03 and eight perturbation examples
- Add three verified literature references and mentor-specific holdout results
- Disclose AI assistance and explain the code independently